

Written Brief Analysis -- Vulnerable Node

Project: Vulnerable Node (cr0hn/vulnerable-node) **Date:** 2026-02-24 **Authors:** Team 17001969 / 17001625

1. Context Map vs. Code Structure: Alignment and Divergences

Alignment

The Context Map identifies three bounded contexts: **IAM**, **Product Catalog**, and **Sales (Orders)**. IAM maps cleanly to dedicated files (`model/auth.js`, `routes/login.js`, `routes/login_check.js`), producing a one-to-one correspondence between the logical domain boundary and the module structure. The supporting infrastructure context maps directly to `config.js`, `model/init_db.js`, and `dummy.js`.

Divergences

The critical divergence occurs between **Catalog** and **Sales**. Both contexts physically coexist inside `routes/products.js` (14 anonymous functions) and `model/products.js` (which exposes `list_products`, `getProduct`, `search`, `purchase`, and `get_purchased` in a single module). There is no file-level or namespace-level boundary separating these domains.

Why These Divergences Exist

The root cause is the monolithic Express.js convention of grouping routes by URL prefix (`/products`) rather than by business capability. The application was designed as a security demonstration tool, not as a production system, so architectural separation was never a priority.

2. Cyclomatic Complexity: Utility and Limitations

Why It Is Useful

Cyclomatic complexity (CC) quantifies independent execution paths, making it a proxy for test effort: $CC = N$ requires at minimum N test cases for full branch coverage. Our CI pipeline enforces $CC \leq 10$ per function. The highest-complexity function (`routes/`

`products.js`, line 89, CC = 7) is indeed the hardest to reason about -- nested conditionals for cart validation, email checking, and purchase branching.

A Concrete Case Where the Metric Is Misleading

`model/auth.js`'s `do_auth` has CC = 1 -- the lowest possible value:

```
function do_auth(username, password) {  
  var q = "SELECT * FROM users WHERE name = '" + username  
    + "' AND password = '" + password + "';";  
  return db.one(q);  
}
```

By CC, this function is trivially simple. In reality, it is the **most dangerous function in the codebase**: it constructs SQL via string concatenation, enabling authentication bypass through SQL injection (`' OR '1'='1`). CC is blind to **semantic risks** -- injection vulnerabilities, insecure API usage, and missing input validation. This demonstrates CC must be complemented with SAST, code review checklists, and dependency auditing.

3. Refactoring Strategy: Why Strangler Fig

We selected the **Strangler Fig** pattern to incrementally extract the three entangled bounded contexts into a `controllers/` + `services/` architecture, without a full rewrite or feature freeze.

Why Strangler Fig Over Alternatives

1. **The application is functional and deployed.** A Big Bang rewrite would introduce regression risk. Strangler Fig allows route-by-route migration: intercepting `/login` through a new `AuthController` while leaving `/products` on the legacy handler, then migrating incrementally.
2. **Clear "seams" at the Express router level.** Each `router.get` / `router.post` can be individually redirected to a new controller. The new "vine" grows alongside the old "tree", intercepting traffic endpoint by endpoint until the legacy code is fully bypassed.
3. **The God-Object `model/products.js` cannot be safely split in one step.** Branch-by-Abstraction would require introducing interfaces in a codebase that lacks any. Strangler Fig avoids

this: each new service (`CatalogService` , `OrderService`) is built independently with parameterized queries, replacing legacy functions one by one.

Risk Mitigation

The CI pipeline (98.38% statement coverage, CC <= 10) provides a safety net. The `OrderService` introduces an Anti-Corruption Layer (ACL) to resolve the price manipulation vulnerability -- querying the real price from the database instead of trusting client-provided values.

4. Biggest Obstacle for a New Engineer

A new engineer faces a **non-functional application on first launch** -- a Time-to-Hello-World measured in hours, not minutes. The obstacle is a chain of three compounding issues:

1. **Silent misconfiguration.** `config.js` defaults to a hardcoded private IP (`10.211.55.70`) belonging to the original developer's VM. Without setting `STAGE=LOCAL` , the app silently times out with no diagnostic message. There is no `.env.example` and no documentation of required variables.
2. **Inverted database initialization.** `model/init_db.js` contains a logic inversion: `.then()` (table created) performs no user seeding, while `.catch()` (table exists) is where insertion occurs. On first startup, the `users` table is created empty; login fails with an opaque error. A second restart is required to populate users.
3. **No governance documentation.** No `README` covering environment variables, no `npm run seed` , no CI validation. Session secret is hardcoded in `app.js` , credentials are in plaintext in `config.js` , and DDL is coupled to the boot cycle.

The combined effect: a new engineer must reverse-engineer configuration, discover the `init_db` bug, and manually fix seed data -- all before writing a single productive line. This is the direct consequence of missing governance and idempotent setup scripts.